Lab Programs-Data Structures

1. Implement basic array operations: insertion, deletion, searching, and traversal.

```
#include <stdio.h>

int main() {
   int arr[100], n, i, pos, value, choice, found;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter elements:\n");
   for(i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }
   do {
      printf("\n--- MENU ---\n");
      printf("1. Traversal\n2. Insertion\n3. Deletion\n4. Searching\n5. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);
      switch(choice) {
         case 1: // Traversal
            printf("Array elements are: ");
            for(i = 0; i < n; i++) {
               printf("%d ", arr[i]);
            }
            break;

         case 2: // Insertion
            printf("Enter position and value: ");
            scanf("%d %d", &pos, &value);

            for(i = n; i >= pos; i--) {
               arr[i] = arr[i - 1];
            }
```

```
            arr[pos - 1] = value;
            n++;
            break;

        case 3: // Deletion
            printf("Enter position to delete: ");
            scanf("%d", &pos);
            for(i = pos - 1; i < n - 1; i++) {
                arr[i] = arr[i + 1];
            }
            n--;
            break;
        case 4: // Searching
            printf("Enter element to search: ");
            scanf("%d", &value);
            found = 0;
            for(i = 0; i < n; i++) {
                if(arr[i] == value) {
                    printf("Element found at position %d\n", i + 1);
                    found = 1;
                    break;
                }
            }

            if(found == 0)
                printf("Element not found\n");
            break;
        case 5:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice!\n");
    }
} while(choice != 5);
```

```
    return 0;
}
```

Output-

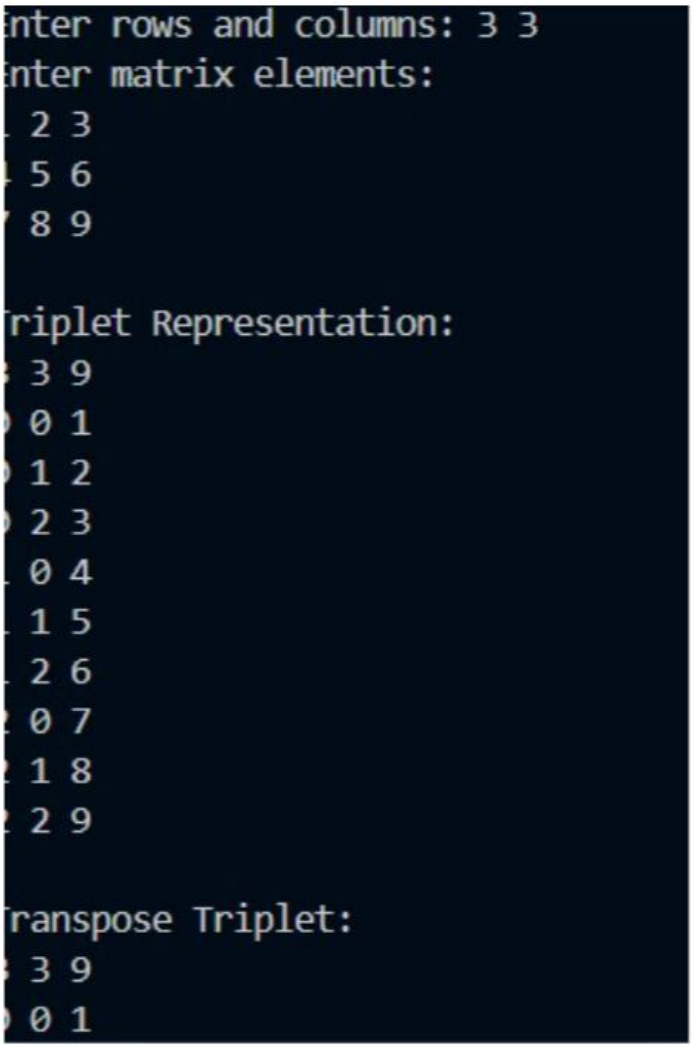
```
Enter elements:
1 2 3 4 5

--- MENU ---
1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
Enter your choice: 2
Enter position and value: 1 56

--- MENU ---
1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
Enter your choice: 4
Enter element to search: 56
Element found at position 1
```

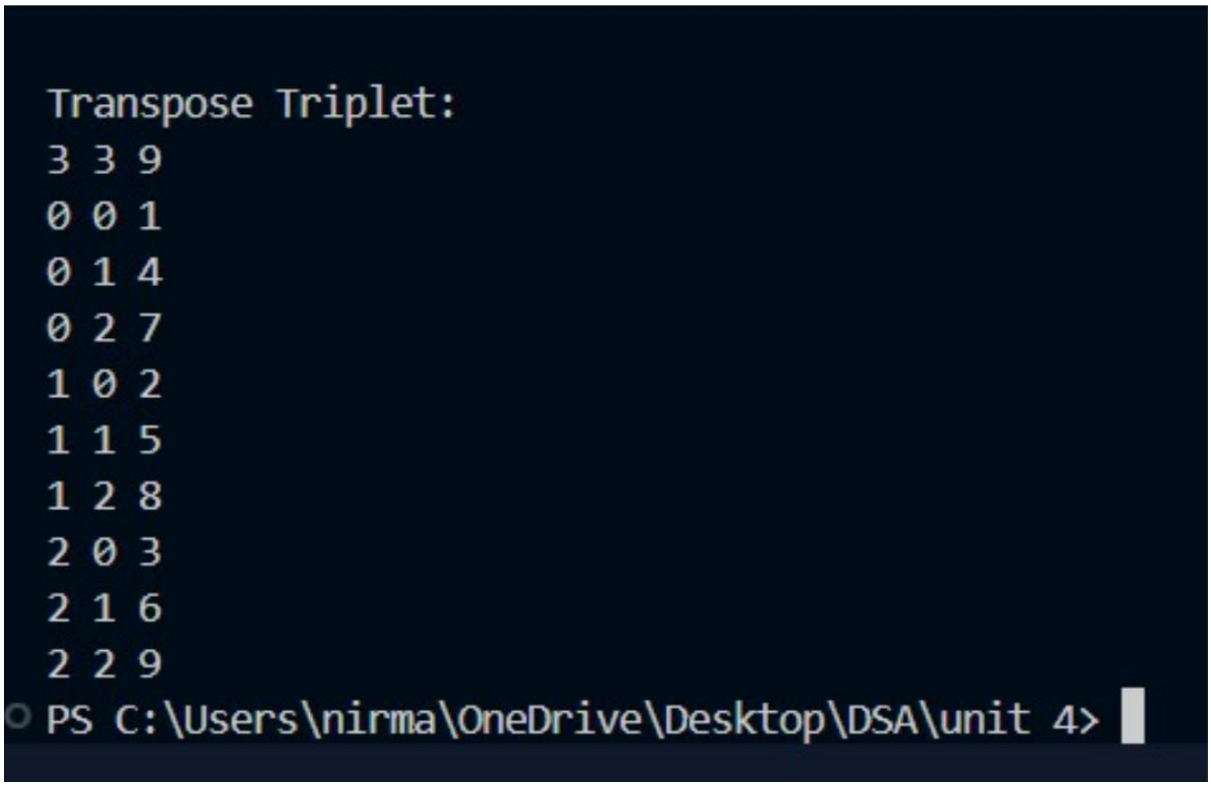
2. Represent a sparse matrix using triplet form and perform transpose operation.

```
#include <stdio.h>
int main() {
    int matrix[10][10], triplet[50][3], transpose[50][3];
    int i, j, m, n, k = 1;
    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);
    printf("Enter matrix elements:\n");
    for(i = 0; i < m; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
```

```
}

// Convert to triplet form
triplet[0][0] = m;
triplet[0][1] = n;
triplet[0][2] = 0;

for(i = 0; i < m; i++) {
   for(j = 0; j < n; j++) {
      if(matrix[i][j] != 0) {
         triplet[k][0] = i;
         triplet[k][1] = j;
         triplet[k][2] = matrix[i][j];
         k++;
         triplet[0][2]++;
      }
   }
}

printf("\nTriplet Representation:\n");
for(i = 0; i < k; i++) {
   printf("%d %d %d\n", triplet[i][0], triplet[i][1], triplet[i][2]);
}

// Transpose
transpose[0][0] = triplet[0][1];
transpose[0][1] = triplet[0][0];
transpose[0][2] = triplet[0][2];

k = 1;

for(i = 0; i < n; i++) {
   for(j = 1; j <= triplet[0][2]; j++) {
      if(triplet[j][1] == i) {
```

```
            transpose[k][0] = triplet[j][1];
            transpose[k][1] = triplet[j][0];
            transpose[k][2] = triplet[j][2];
            k++;
        }
    }
  }
  printf("\nTranspose Triplet:\n");
  for(i = 0; i < k; i++) {
    printf("%d %d %d\n", transpose[i][0], transpose[i][1], transpose[i][2]);
  }
  return 0;
}
```





3. lab program 3

```
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

// Push
void push(int value) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else {
        top++;
        stack[top] = value;
        printf("Pushed %d\n", value);
    }
```

```
}

// Pop
void pop() {
    if (top == -1)
        printf("Stack Underflow\n");
    else {
        printf("Popped %d\n", stack[top]);
        top--;
    }
}

// Display
void display() {
    if (top == -1)
        printf("Stack is empty\n");
    else {
        printf("Stack elements:\n");
        for (int i = top; i >= 0; i--)
            printf("%d\n", stack[i]);
    }
}

// Main
int main() {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
```

}

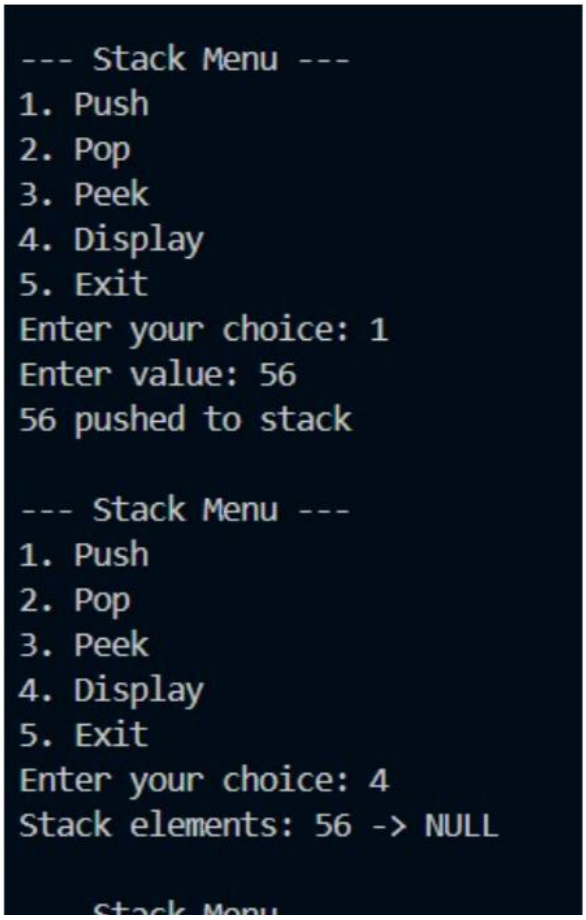
```
Pushed 10
Pushed 20
Pushed 30
Stack elements:
30
20
10
Popped 30
Stack elements:
20
10
PS C:\Users\nirma\OneDrive\Desktop\DSA\unit 4>
```

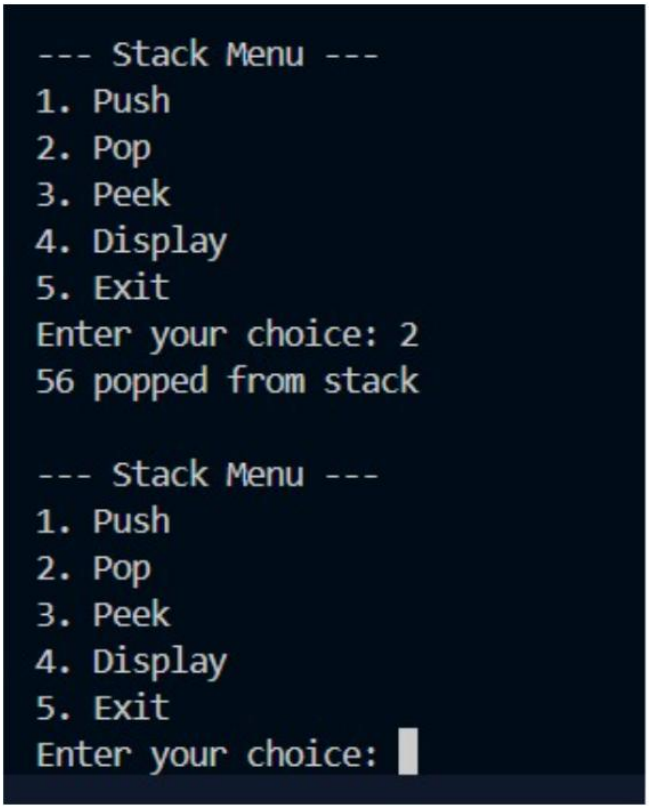
4. lab program 4

```
#include <stdio.h>
#include <stdlib.h>

// Define node structure
struct Node {
    int data;
    struct Node* next;
};
struct Node* top = NULL;  // Top of stack
// Push operation
void push(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Stack Overflow (Memory not available)\n");
        return;
    }

    newNode->data = value;
    newNode->next = top;
    top = newNode;

    printf("%d pushed to stack\n", value);
}
```

```
// Pop operation
void pop() {
    if (top == NULL) {
        printf("Stack Underflow (Empty stack)\n");
        return;
    }
    struct Node* temp = top;
    printf("%d popped from stack\n", top->data);

    top = top->next;
    free(temp);
}

// Peek operation
void peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
    } else {
        printf("Top element is: %d\n", top->data);
    }
}

// Display stack
void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return;
    }

    struct Node* temp = top;
    printf("Stack elements: ");

    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    int choice, value;
```

```
    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```


--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 56
56 pushed to stack
--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements: 56 -> NULL


--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
56 popped from stack
--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:

5. LAB PROGRAM 5

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top >= MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = c;
}

char pop() {
    if (top < 0) {
        return '\0';
    }
    return stack[top--];
}

char peek() {
    if (top < 0) {
        return '\0';
    }
    return stack[top];
}

int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
```

```
            return 3;
        default:
            return 0;
    }
}
int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}
void infixToPostfix(char *infix, char *postfix) {
    int i = 0, j = 0;
    char c;
    while ((c = infix[i++]) != '\0') {
        // If operand, add to output
        if (isalnum(c)) {
            postfix[j++] = c;
        }
        // If '(', push to stack
        else if (c == '(') {
            push(c);
        }
        // If ')', pop until '(' is found
        else if (c == ')') {
            while (top >= 0 && peek() != '(') {
                postfix[j++] = pop();
            }
            pop(); // Remove '(' from stack
        }
        // If operator
        else if (isOperator(c)) {
            // Handle right-associativity for '^'
            if (c == '^') {
                while (top >= 0 && precedence(peek()) > precedence(c)) {
                    postfix[j++] = pop();
                }
            } else {
                while (top >= 0 && precedence(peek()) >= precedence(c)) {
                    postfix[j++] = pop();
                }
            }
            push(c);
        }
    }
    // Pop remaining operators
    while (top >= 0) {
```

```
        postfix[j++] = pop();
    }
    postfix[j] = '\0';
}
int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter an infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

```
Enter an infix expression: a+b*c
Postfix expression: abc*+
PS C:\Users\nirma\OneDrive\Desktop\DSA\unit 4>
```

6.LAB PROGRAM 6

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}

void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
```

```
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void insertAtPosition(struct Node** head, int data, int position) {
    struct Node* newNode = createNode(data);
    if (position == 0) {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position out of range\n");
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}

void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

void deleteFromEnd(struct Node** head) {
    if (*head == NULL) return;
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
```

```
    }
    struct Node* temp = *head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range\n");
        return;
    }
    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}

void traverse(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int search(struct Node* head, int target) {
    struct Node* temp = head;
    while (temp != NULL) {
        if (temp->data == target) return 1;
        temp = temp->next;
```

```
    }
    return 0;
}

void reverse(struct Node** head) {
    struct Node* prev = NULL;
    struct Node* current = *head;
    struct Node* next = NULL;
    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head = prev;
}

void sort(struct Node** head) {
    if (*head == NULL || (*head)->next == NULL) return;
    struct Node* temp = *head;
    while (temp != NULL) {
        struct Node* next = temp->next;
        while (next != NULL) {
            if (temp->data > next->data) {
                int data = temp->data;
                temp->data = next->data;
                next->data = data;
            }
            next = next->next;
        }
        temp = temp->next;
    }
}

int length(struct Node* head) {
    int count = 0;
    struct Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

```
int main() {
    struct Node* head = NULL;

    insertAtEnd(&head, 10);
    insertAtEnd(&head, 20);
    insertAtEnd(&head, 30);
    traverse(head);

    insertAtBeginning(&head, 5);
    traverse(head);

    insertAtPosition(&head, 25, 2);
    traverse(head);

    deleteFromBeginning(&head);
    traverse(head);

    deleteFromEnd(&head);
    traverse(head);

    deleteFromPosition(&head, 1);
    traverse(head);

    printf("List contains 20: %s\n", search(head, 20) ? "Yes" : "No");
    printf("List contains 40: %s\n", search(head, 40) ? "Yes" : "No");

    reverse(&head);
    traverse(head);

    insertAtEnd(&head, 15);
    insertAtEnd(&head, 5);
    traverse(head);

    sort(&head);
    traverse(head);

    printf("Length of the list: %d\n", length(head));

    return 0;
}
```

```
}
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4
PS C:\Users\nirma\OneDrive\Desktop\DSA\unit 4>
```